

分享主题：零侵入架构

超大规模k8s集群kube-apiserver内存占用降低50%实战优化

明廷樟

蚂蚁集团开发工程师

2025/11/15





明廷樟

嘉宾职位： 软件工程师

个人简介： 蚂蚁集团 开发工程师

CONTENT

目录

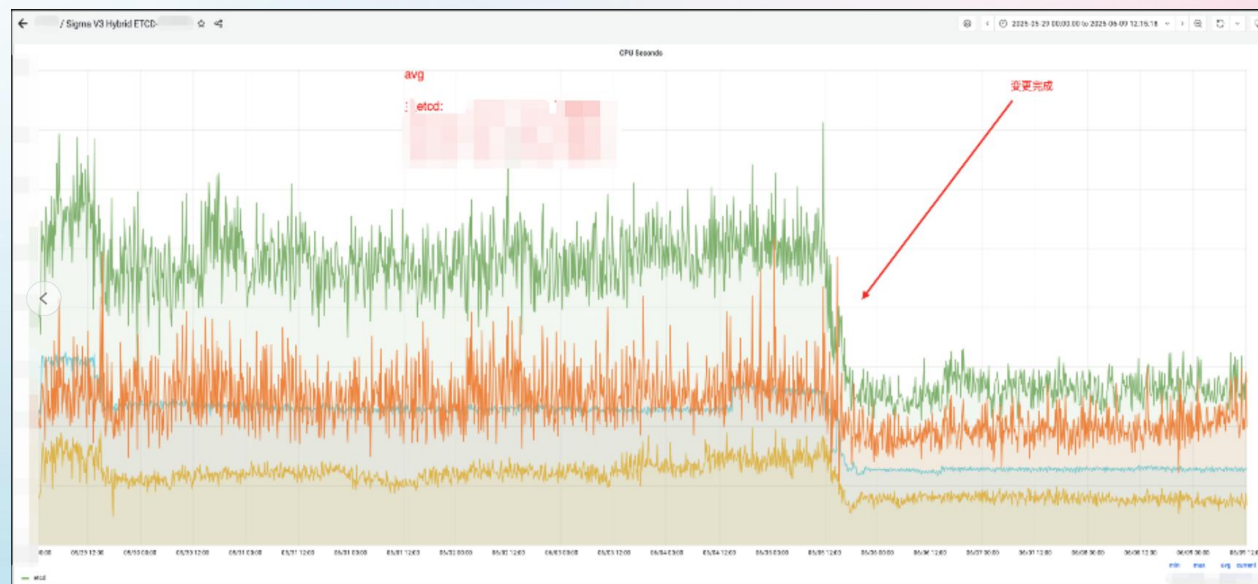
- 01 背景：超大k8s集群的挑战
- 02 方案：架构升级-分而治之
- 03 总结：收益及问题总结
- 04 展望：未来的一些思路

结论先行

效果：apiserver/etcd 性能提升

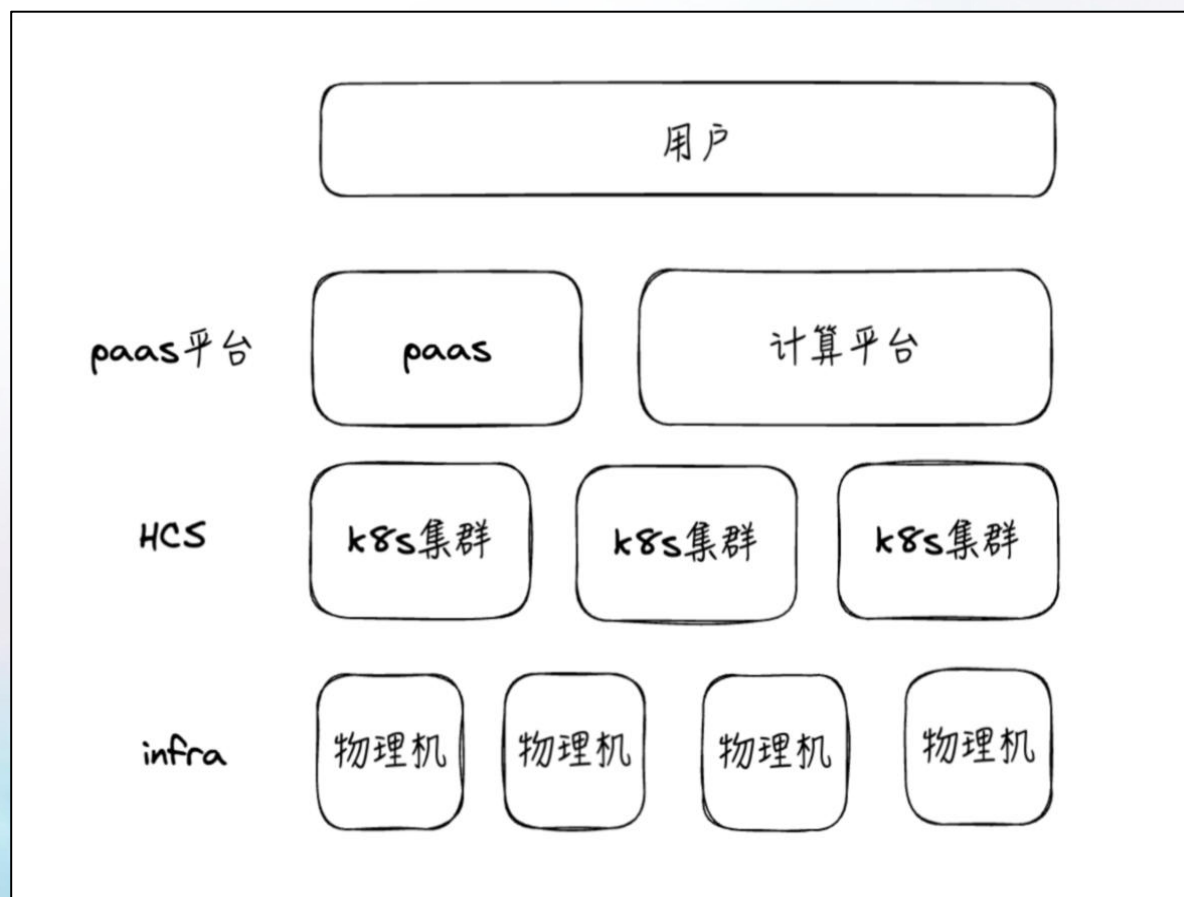
Apiserver: **实例个数不变**的情况下 平均内存下降
50%

Etcd性能: cpu seconds 下降**40%**



背景：超大规模k8s集群挑战

业务影响： kube-apiserver的稳定性对业务的影响



背景：超大规模k8s集群挑战

资源压力： 面临巨大的资源(内存, 单机等)压力

维度	风险
集群规模	集群规模巨大; nodes: 超过2w
APIServer 内存占用	APIServer 单实例OOM , 突发流量导致多实例同时oom, 导致控制面 雪崩
单实例最大承载量	限流阈值与业务交付SLO矛盾: 降低限流影响业务 SLA, 保持现状则牺牲 系统鲁棒性
单机启动耗时长	发布耗时长达 几个小时级别 , 增加 运维负担 增重 (应急操作、应急回滚)

背景：超大规模k8s集群挑战

资源耦合风险：

背景：所有资源类型 (Pod, Service, CRD等)共享Apiserver实例, 资源保障等级不一

风险：非核心资源(Event)挤占核心资源(Pod)处理能力

案例：event流量导致整个集群出现雪崩

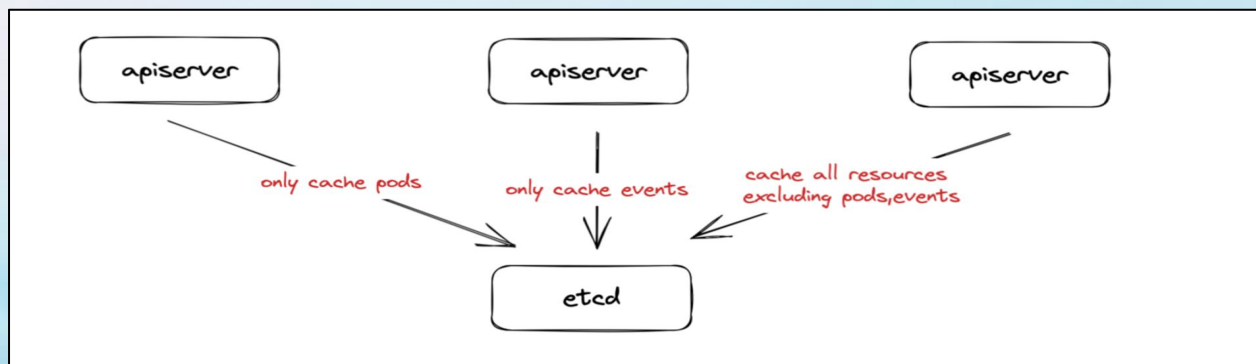
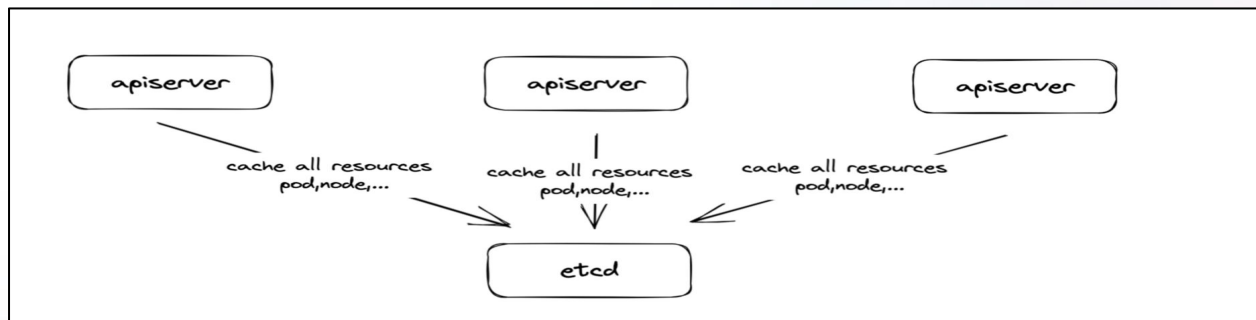
CONTENT

目录

- 01 背景：超大k8s集群的挑战
- 02 方案：架构升级—分而治之
- 03 总结：收益及问题总结
- 04 展望：未来的一些思路

方案：架构升级-分而治之

整体思路：**分而治之**

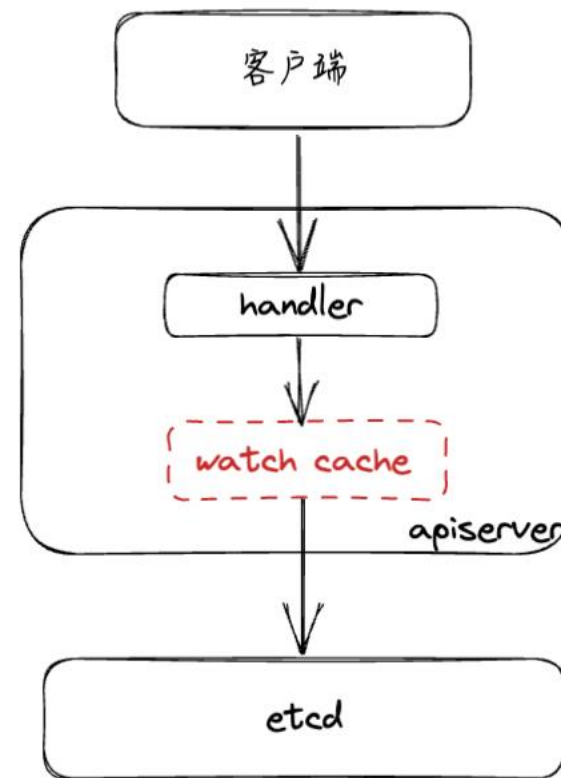


方案：架构升级-分而治之

理论基础：apiserver 内部watch cache逻辑

要点：

1. Watch Cache: apiserver会对所有资源粒度从etcd缓存到本地
2. 逻辑: 如果某个资源没有watch cache, 则会直接打穿到etcd

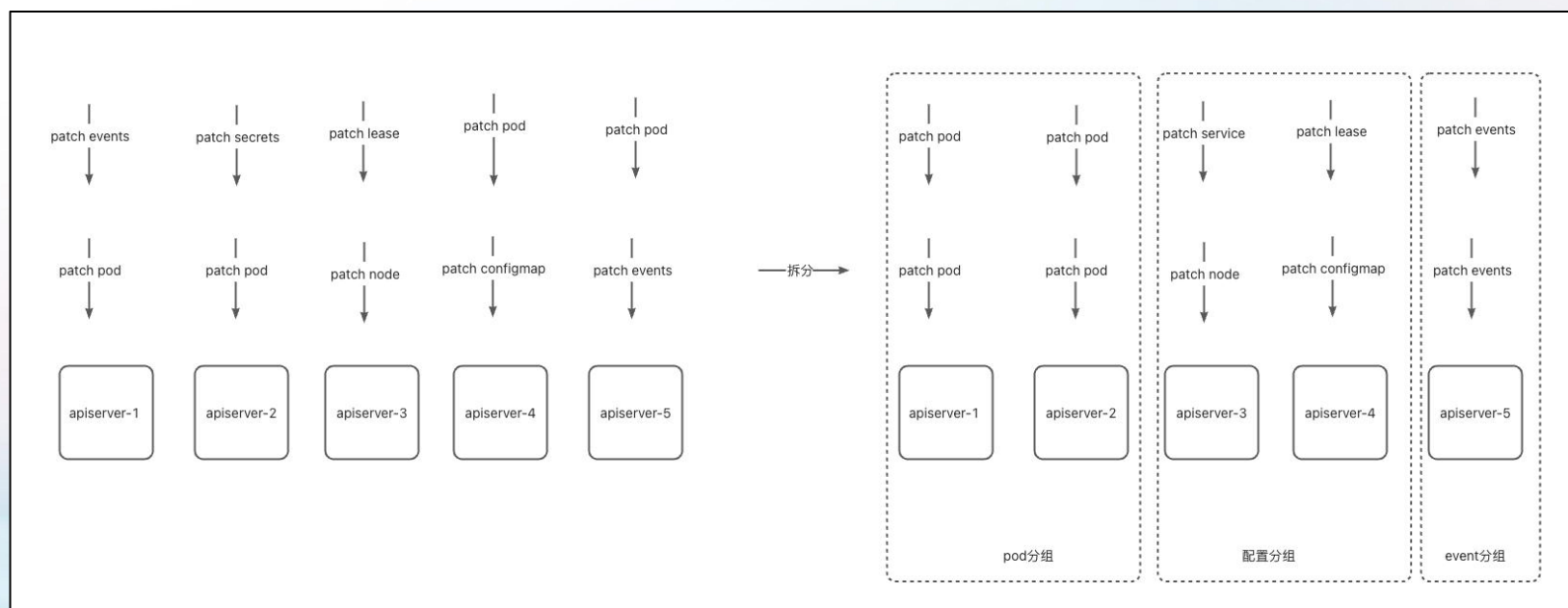


方案：架构升级-分而治之

理论基础：按照资源类别的请求分析

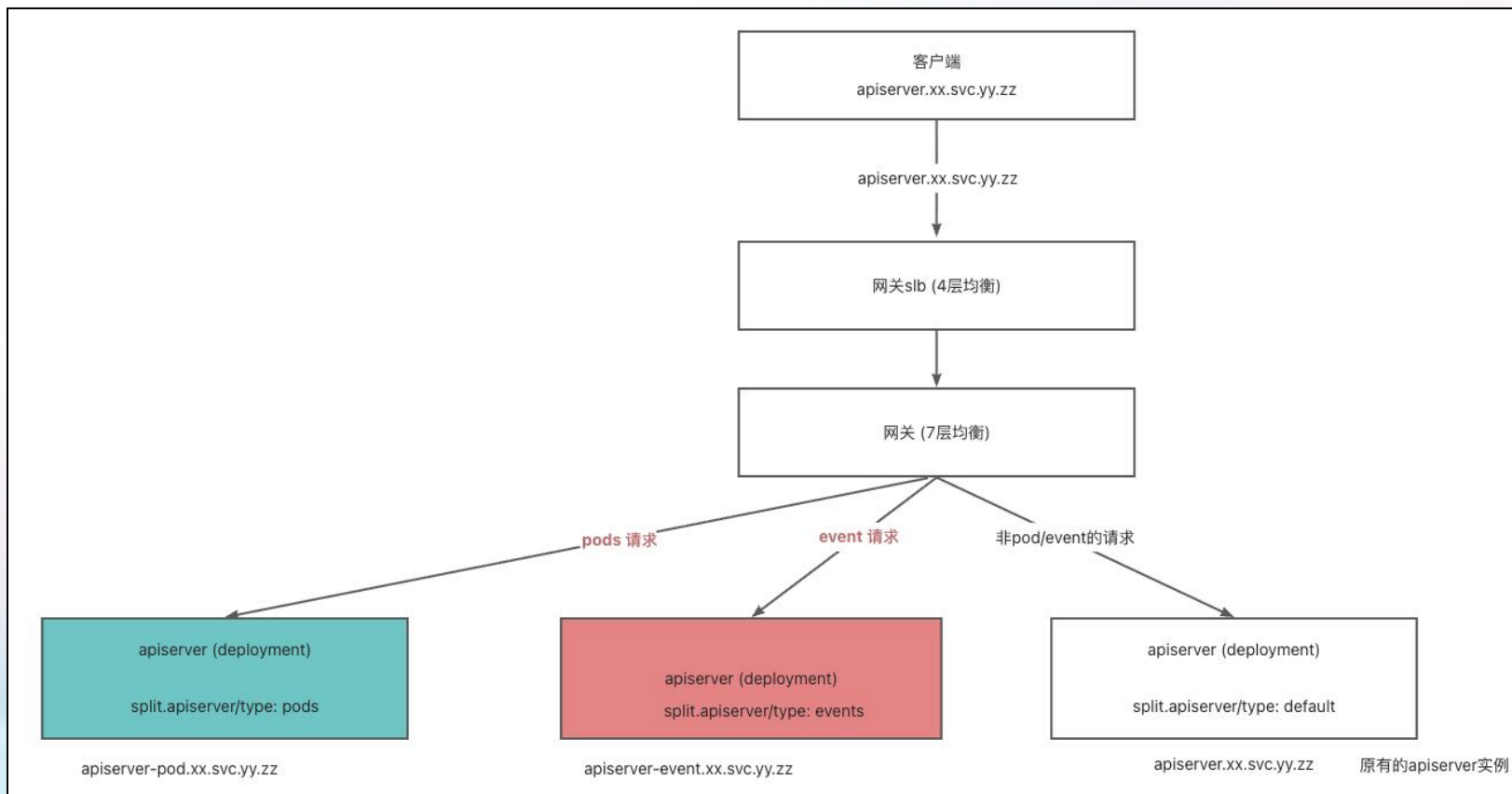
要点：

1. 节约内存资源: pod分组apiserver只需要cache pod资源的数据



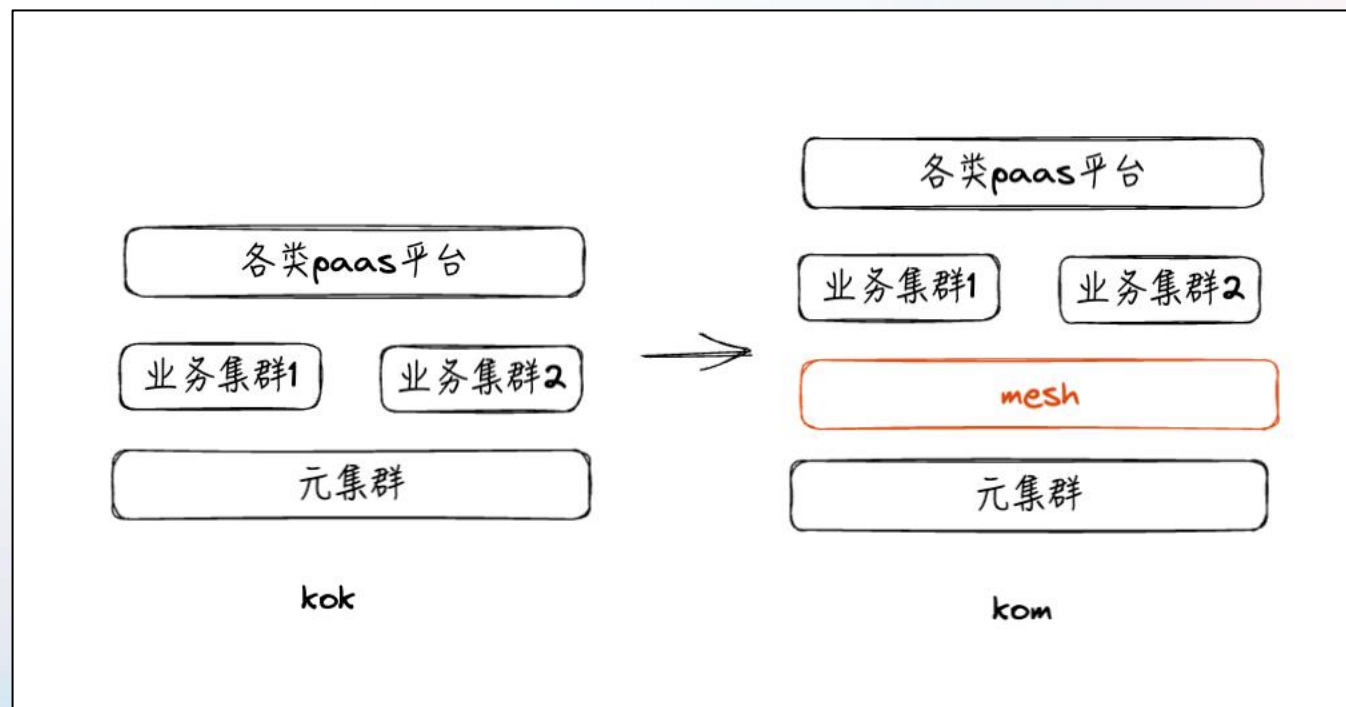
方案：架构升级-分而治之

技术方案：流量按资源进行路由

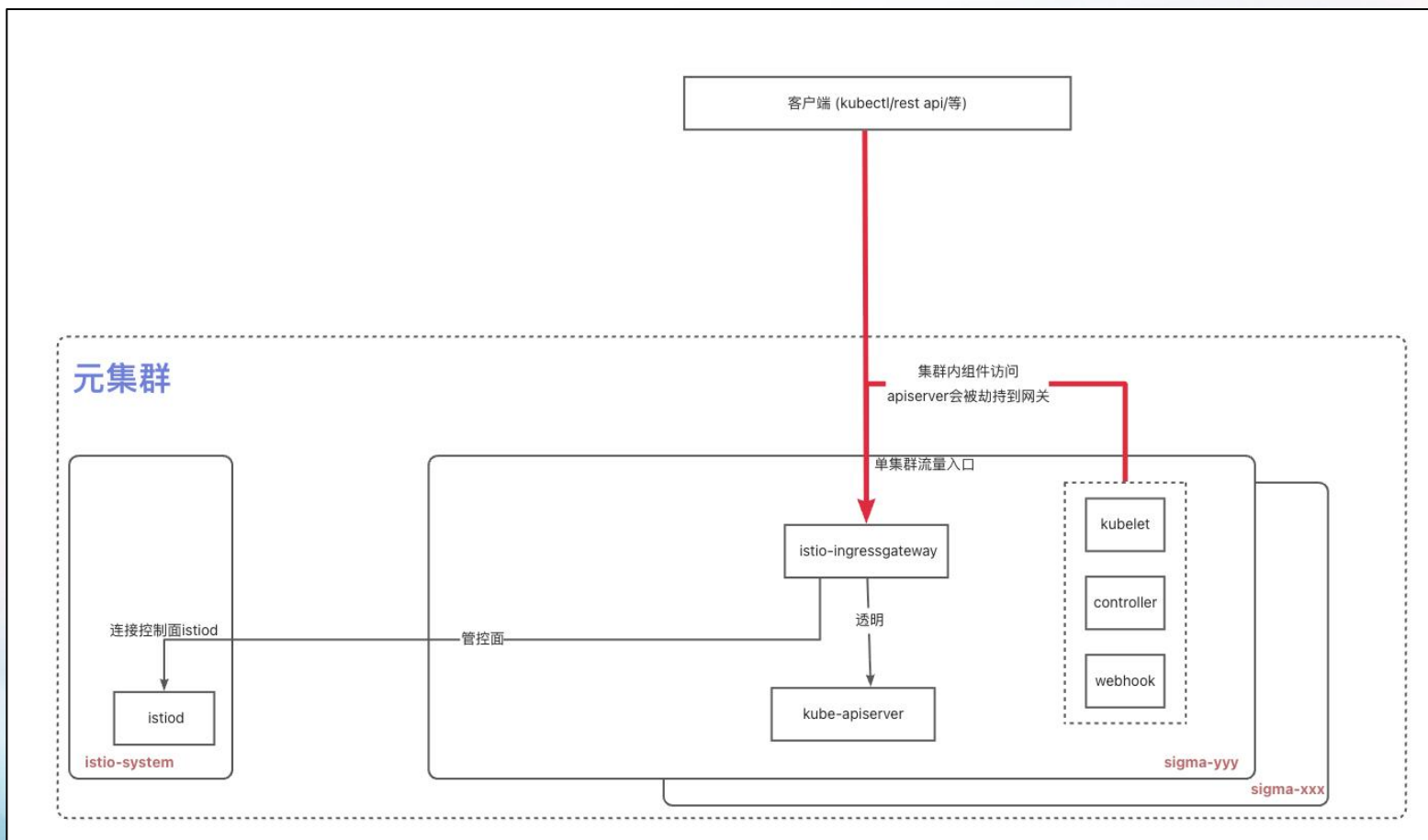


方案：架构升级-分而治之

技术方案：网关 kok (kubernetes on kubernetes) -> kom (kubernetes on mesh)

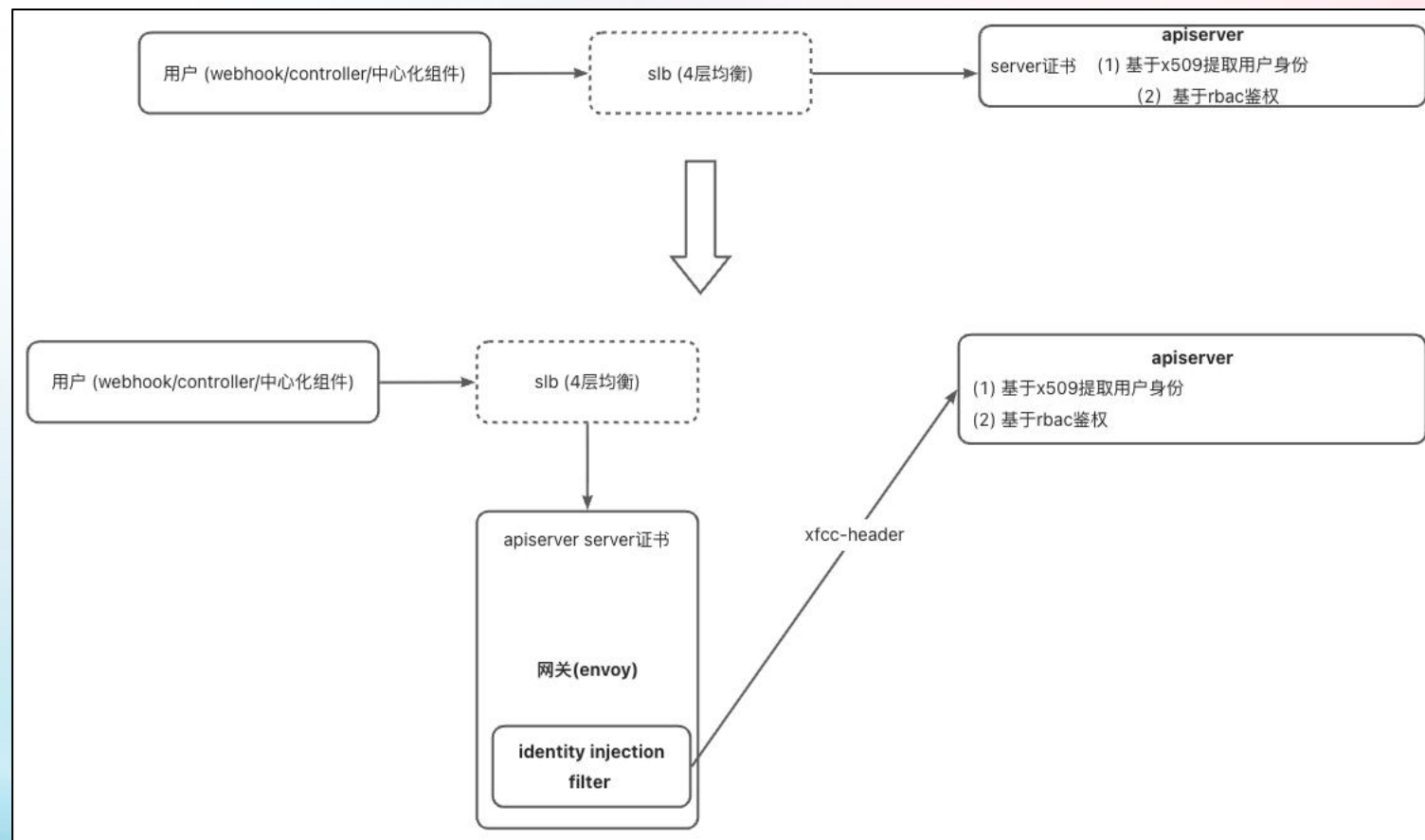
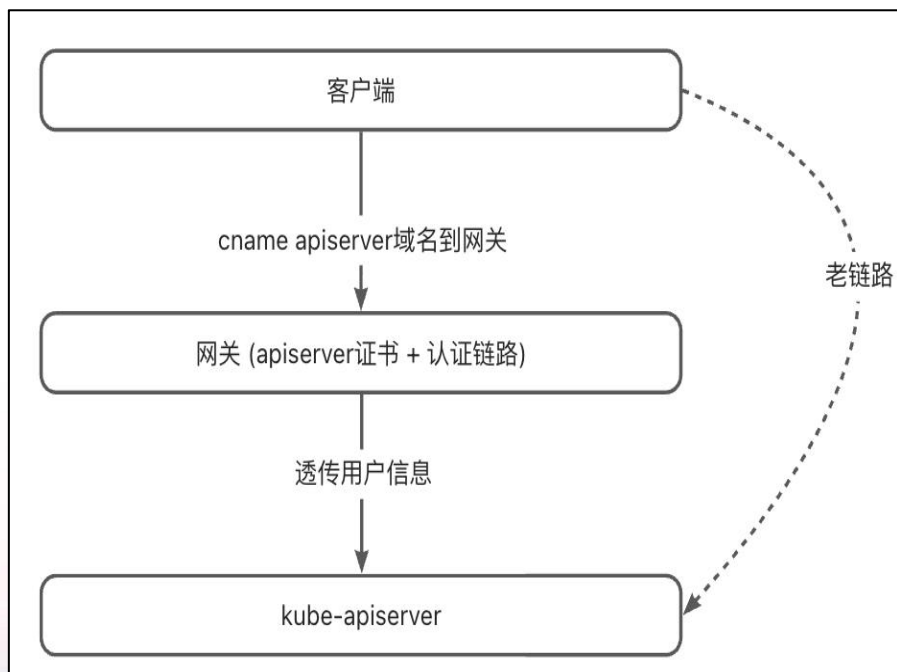


技术方案： kom网关 单集群流量入口



方案：架构升级-分而治之

技术方案： kom网关 **无感升级**



方案：架构升级-分而治之

技术方案：资源拆分分组

分组：

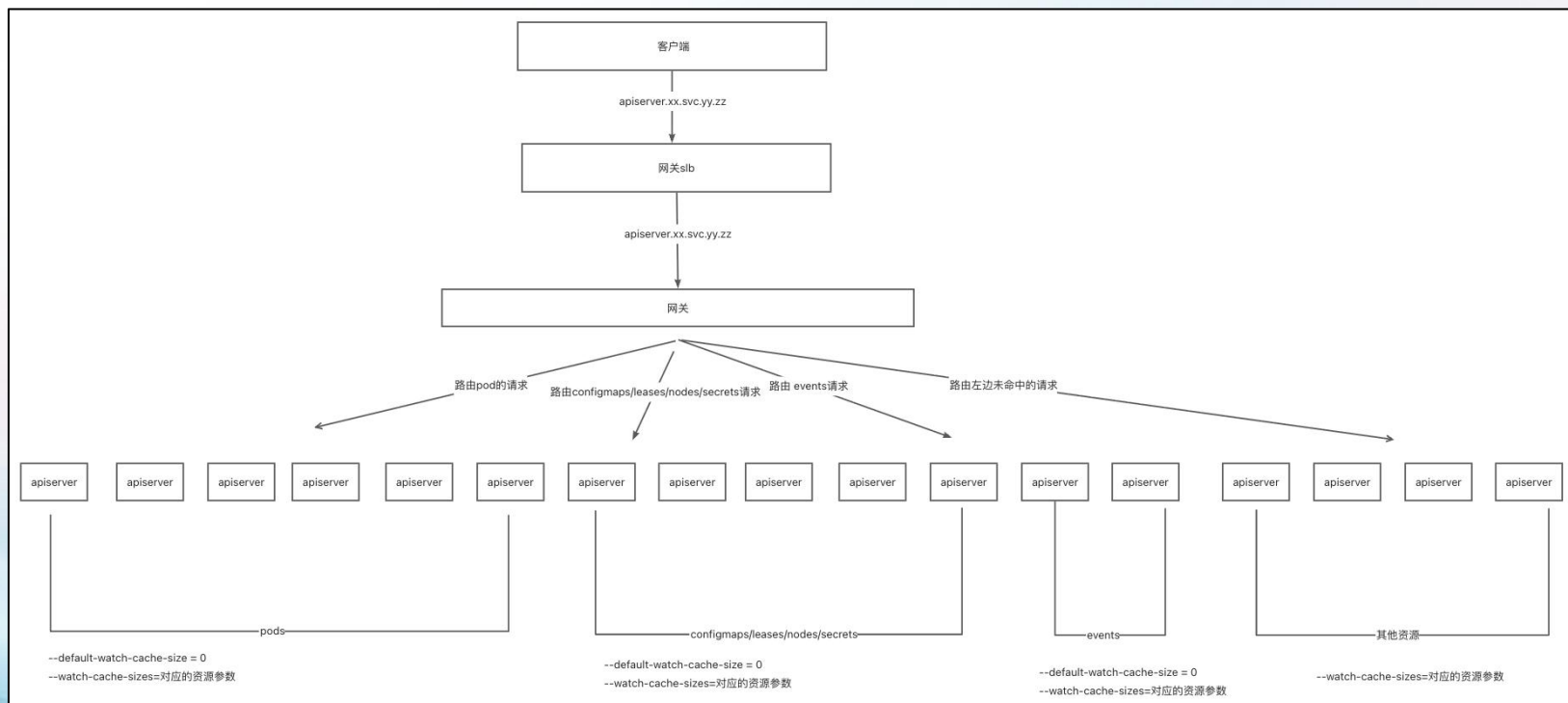
1. Pod：交付核心资源，独占
2. Config分组：配置类资源，包括(nodes/configmaps/leases/secrets)等资源
3. Event：非交付链路资源
4. Default分组：cache所有资源，一旦任何异常，可将流量回切至该分组

方案：架构升级-分而治之

技术方案：apiserver分组

关键参数：

1. **--default-watch-cache-size**: 默认的watch cache size大小
2. **--watch-cache-sizes**: 对应资源的watch cache 个数比如endpoints#0, endpointslices.discovery.k8s.io#0, nodes#0

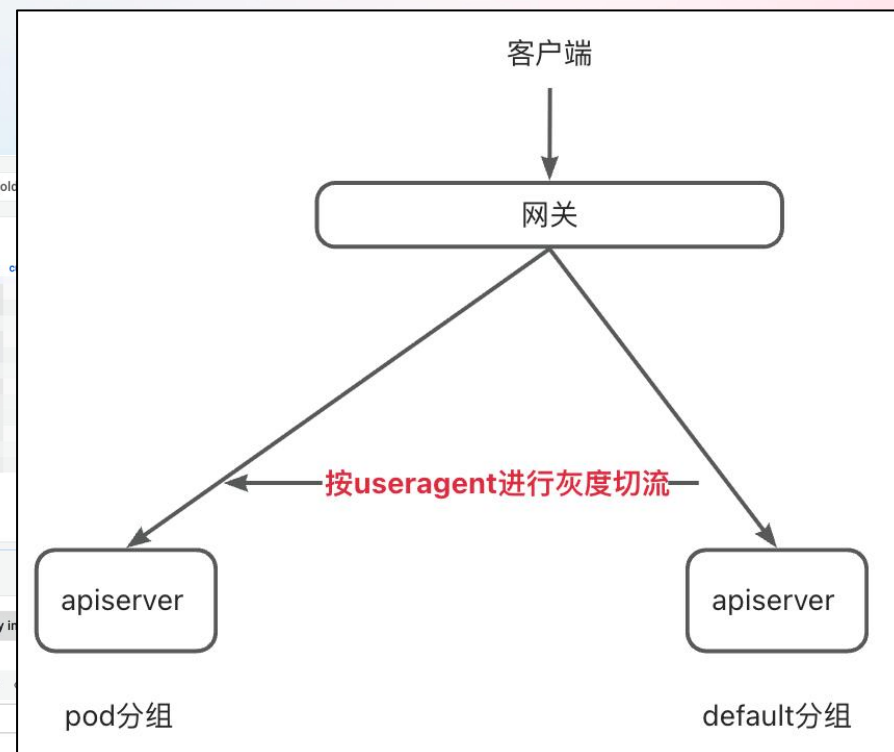
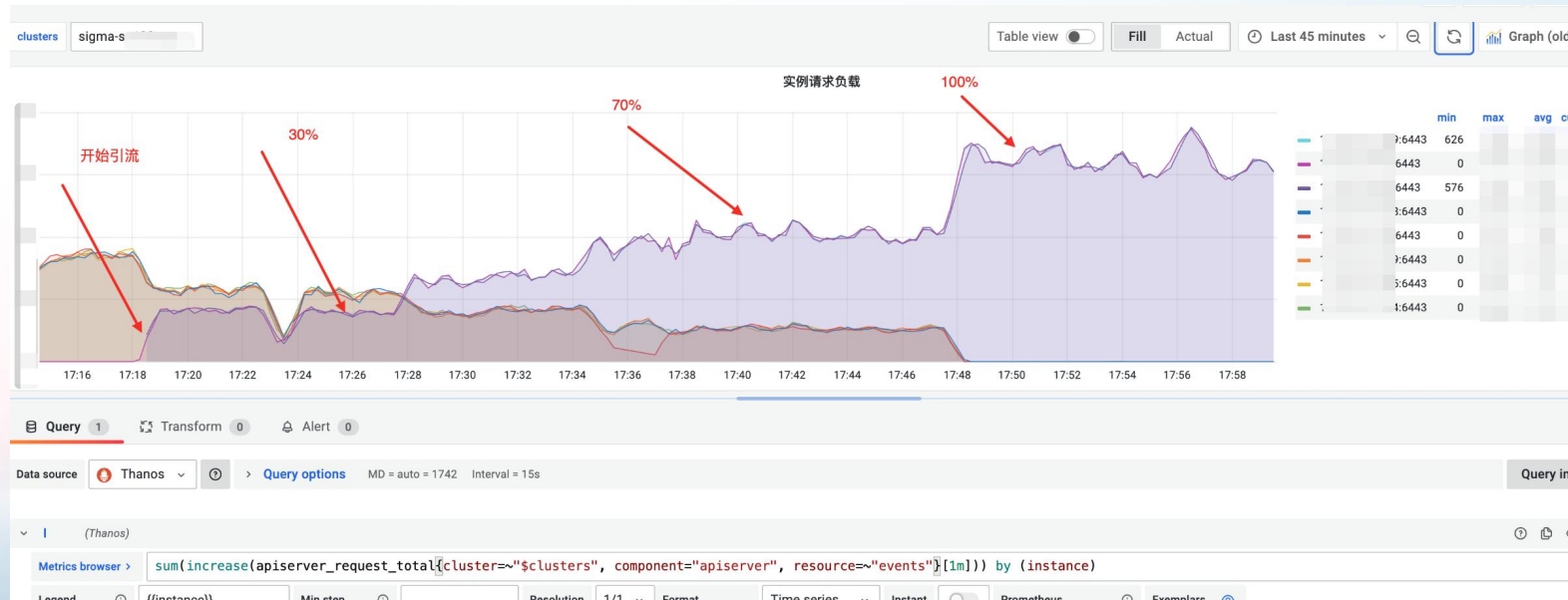


方案：架构升级-分而治之

技术方案：灰度无感丝滑升级

流程：

1. 新建 其他(pod)分组
2. 将流量按照useragent将客户端从原有apiserver(default分组)灰度切流



CONTENT

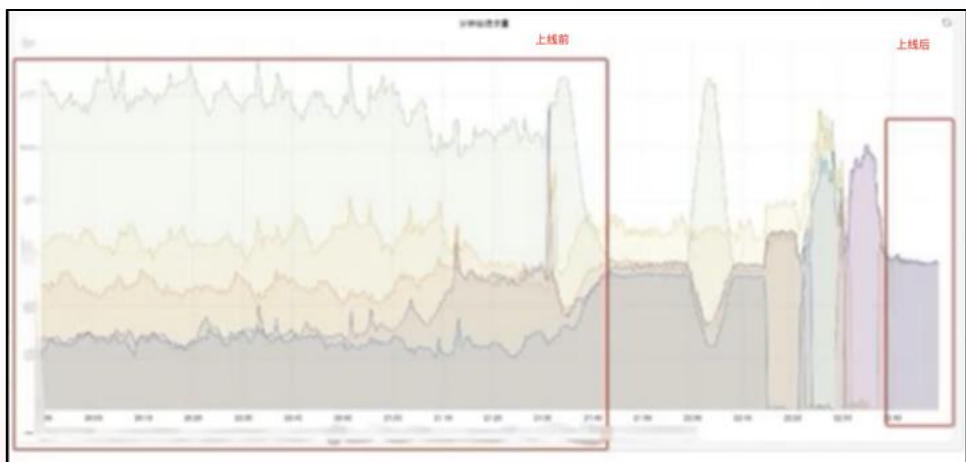
目录

- 01 背景：超大k8s集群的挑战
- 02 方案：架构升级-分而治之
- 03 **总结：收益及问题总结**
- 04 展望：未来的一些思路

总结：收益

效果：kom网关的一些收益

7层流量负载均衡



成功率提升



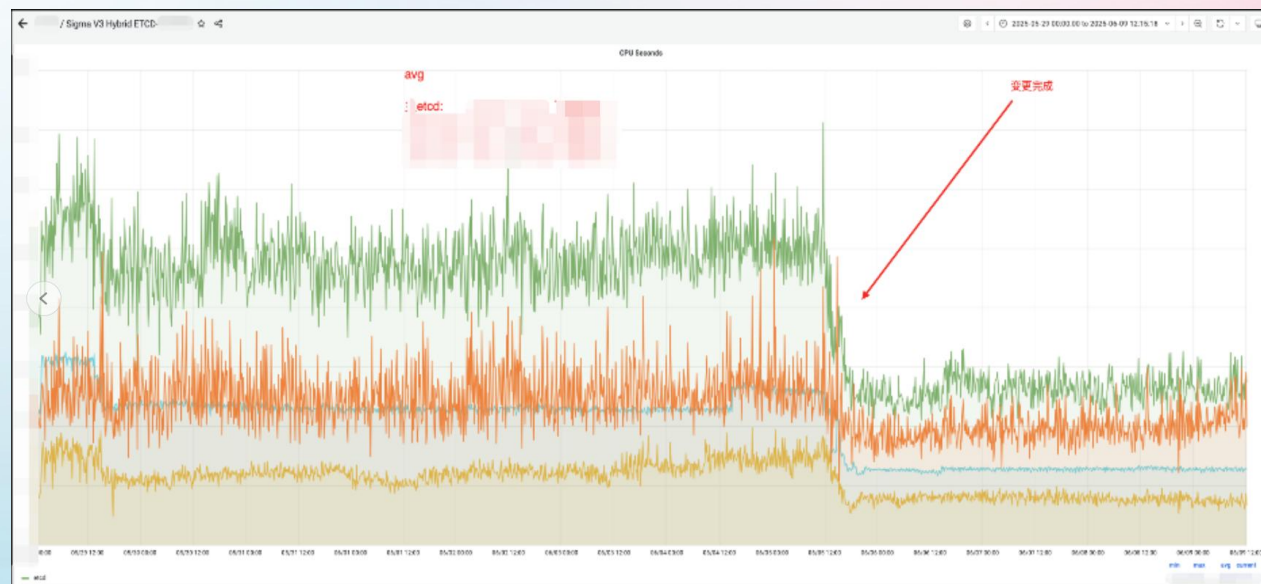
灰度升级



总结：收益

效果：apiserver/etcd性能收益

Apiserver: **实例个数不变**的情况下 平均内存下降**50%** Etcd性能: cpu seconds 下降**40%**



总结： 上线过程中遇到的一些问题

问题1: event apiserver分组从etcd list all pods

原因:

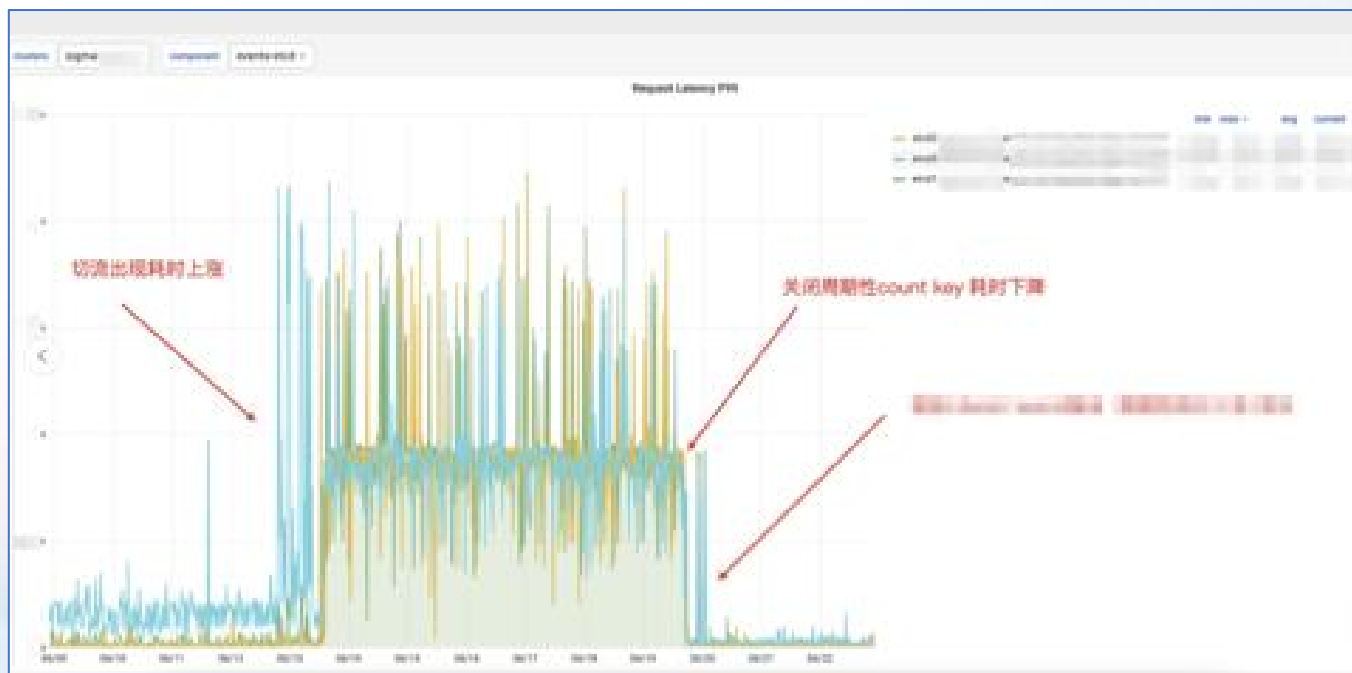
- **node鉴权/TokenRequest featuregate**开启 会导致 list all pods

```
1 /api/v1/endpoints
2 /api/v1/namespaces
3 /api/v1/namespaces/kube-system/configmaps
4 /api/v1/nodes
5 /api/v1/persistentvolumes
6 /api/v1/pods
7 /api/v1/secrets
8 /api/v1/serviceaccounts
9 /api/v1/services
10 /apis/admissionregistration.k8s.io/v1/mutatingwebhookconfigurations
11 /apis/admissionregistration.k8s.io/v1/validatingwebhookconfigurations
12 /apis/apiextensions.k8s.io/v1/customresourcedefinitions
13 /apis/apiregistration.k8s.io/v1/apiservices
14 /apis/rbac.authorization.k8s.io/v1/clusterrolebindings
15 /apis/rbac.authorization.k8s.io/v1/clusterroles
16 /apis/rbac.authorization.k8s.io/v1/rolebindings
17 /apis/rbac.authorization.k8s.io/v1/roles
18 /apis/storage.k8s.io/v1/volumeattachments
```

总结： 上线过程中遇到的一些问题

问题2： apiserver会周期性调用etcd 进行 count key

解决： `--etcd-count-metric-poll-period=0`



总结： 上线过程中遇到的一些问题

问题3： kom网关上线遇到的一些问题

1. 网关Stream error 偶发断连问题

- 调整keep alive 配置

1. H2 flow control 导致的部分请求rt

- 调整H2流控配置

CONTENT

目录

- 01 背景：超大k8s集群的挑战
- 02 方案：架构升级—分而治之
- 03 总结：收益及问题总结
- 04 展望：未来的一些思路

展望

要点:

1. 将pod资源彻底从不需要的分组中去除掉,会进一步提升apiserver/etcd性能
2. Controller类组件同样面临巨大的内存压力, 缺乏水平扩展能力

谢谢观看

